

Traducción estadística basada en frases: MOSES

Miguel García Bohigues

Diciembre 2022

1 Introducción

El objetivo de esta práctica consiste en construir sistemas de traducción basados en modelos estadísticos formados a partir de pares de frases bilingües. Para la construcción de estos modelos vamos a utilizar la herramienta *Moses*.

Moses es un sistema que permite entrenar modelos de traducción automática para cualquier par de lenguas de entrada y de salida. Dados un conjunto de ficheros pares de entrada y salida, *Moses* entrena un modelo sobre el que luego proporciona un algoritmo de búsqueda que permite encontrar rápidamente la traducción de máxima probabilidad.

Moses es una herramienta cuya última versión fue lanzada en octubre del 2017. Para un software actual este es un periodo de tiempo muy elevado sin disponer de una nueva versión. No obstante dispone tanto de repositorio de git, como de un manual que ayuda a su uso e instalación.

Para el caso concreto del autor de esta memoria, se ha optado por descargar los ficheros fuente de la solución para compilarlos en la propia máquina en la que se ha realizado la práctica. Esto ha requerido instalar tanto *Moses*, como sus dependencias. Así como *Giza* y *SRILM*.

Una vez instalados todos los softwares necesarios la guía de la práctica comenta como construir un sistema de traducción paso por paso, desde la preparación de los datos de entrada hasta la evaluación de los resultados. Para ello se ha utilizado el *BLEU*, un método de evaluación que presenta un valor más elevado cuanto más similar es la frase producida con respecto a otra de referencia.

Los datos utilizados han sido un conjunto de la tarea *EuTrans* que contiene ficheros en castellano e inglés correspondiente a las transcripciones de escenarios en los que un turista se encuentra frente al mostrador de un hotel.

El modelo generado mediante la guía para los datos presentados ha conseguido un *BLEU* de 92.12, 97.6/93.2/91.3/89.3 (BP=0.993, ratio=0.993, hyp.len=11700, ref.len=11786). Tal y como muestra la memoria.

2 Ejercicios

2.1 Ejercicio 1

Esta actividad propone experimentar con el modelo desarrollado. En concreto se propone prescindir de un ajuste de pesos para el entrenamiento del modelo. Esto se puede hacer realizando el proceso de traducción directamente con el modelo de traducción que obtenemos con *GIZA++*.

En la siguiente tabla se muestra el resultado obtenido con este modelo comparándolo con el resultado obtenido con el modelo de ejemplo.

Modelo	BLEU
Modelo de ejemplo	92.12
Modelo sin ajuste	88.65

Table 1: Comparación de modelos en función del ajuste de pesos

Como vemos, renunciar al ajuste de pesos del modelo log-lineal nos ofrece más bien poco. Siendo que no reducimos significativamente el coste temporal de generación del modelo y el resultado que obtenemos es casi 4 puntos de *BLEU* inferior al modelo base.

2.2 Ejercicio 2

El siguiente apartado propone experimentar con el número máximo de iteraciones para entrenar el modelo log-lineal.

En el modelo base se ha fijado este valor a un máximo de 5, para evitar un entrenamiento muy prolongado en el tiempo. Sabiendo esto, vamos a proceder a incrementar el valor para ver cómo se comporta el modelo con los diferentes valores.

Para poder realizar este proceso, y todos los que siguen, de manera más rápida y automática, se ha desarrollado un shell-script que automatiza todas las acciones a llevar a cabo para crear y entrenar todo el sistema de traducción. En el anexo de la presente memoria se puede ver el código desarrollado.

Los resultados obtenidos han sido los siguientes:

Max_iter	BLEU
5	92.12
7	92.01
8	92.02
10	91.67

Table 2: Evolución del BLEU en función del valor máximo de iteraciones en MERT

Como vemos, el valor no varía sustancialmente. No podemos afirmar que un número máximo de iteraciones sea más adecuado que otro, debido a que este valor representa el número máximo de iteraciones sin convergencia.

2.3 Ejercicio 3

Siguiendo con la experimentación para intentar mejorar este modelo de traducción, se sugiere probar con diferentes valores de n-gramas a la hora de generar el modelo de lenguaje de salida, en este caso inglés.

Para ello, al script desarrollado en el punto anterior se le ha añadido la parametrización del parámetro *NGRAM*, que indica el tamaño del ngrama a tener en cuenta para la generación del modelo del lenguaje de salida.

Para el experimento se han probado longitudes de n-grama desde 1 hasta 7. La siguiente tabla muestra los resultados.

Lamentablemente para tamaño de n-grama de 1 y 7 no se ha conseguido obtener resultado. A la hora de entrenar los pesos del modelo log-lineal *Moses* devolvía un error que no se supo solucionar. No obstante, para los valores comprendidos entre 2 y 6 vemos que el que mejor resultado de *BLEU* ha reflejado

Tamaño de n-grama	BLEU
n = 1	MOSES_ERR
n = 2	91.57
n = 3	92.12
n = 4	92.41
n = 5	92.48
n = 6	92.24
n = 7	MOSES_ERR

Table 3: Evolución del BLEU en función del tamaño de Ngrama

es $n = 5$. Aunque en ningún caso llegamos a mejorar notablemente el bleu obtenido por el modelo base.

2.4 Ejercicio 4

Para el último de los ejercicios obligatorios del boletín, se propone probar *MIRA* como método para el entrenamiento de los pesos del modelo log-lineal.

Esto se puede conseguir mediante el comando *-batch-mira* a la hora de realizar el entrenamiento de dicho modelo. Adicionalmente, se puede agregar un indicador que permite parametrizar el número de veces que se recorren los datos. Recorrer más veces los datos puede resultar beneficioso para datasets que sean más pequeños. Los resultados que hemos obtenido se pueden ver en la tabla siguiente:

Modelo	BLEU
Modelo base	92.12
Mira, J=60	90.88
Mira, J=100	90.91
Mira, J=200	90.95

Table 4: BLEU obtenido en función del metodo de entrenamiento del modelo log-lineal

Como vemos, utilizar *MIRA*, no mejora el modelo base. No obstante vemos que, en el uso de mira, incrementar el número de veces que se recorren los datos sí incrementa el valor obtenido del *BLEU*, aunque la diferencia es ínfima.

2.5 Ejercicio 5 (opcional)

Hasta el momento, en todos los experimentos que hemos realizado hemos utilizado el descuento de Knesser-ney para el suavizado del modelo de n-gramas. En esta sección vamos a probar otros modelos de suavizado para observar cómo afectan al resultado obtenido.

Para probar los diferentes descuentos, basta con modificar la instrucción que genera el modelo de ngramas, pasándole el descuento que queramos utilizar.

Suavizado	BLEU
Unmod. Knesser-ney	91.76
Witten-bell and interpolation	91.63
Knesser-ney	92.21

Table 5: Bleu obtenido dependiendo del suavizado aplicado al modelo de N-gramas

en nuestro caso ha sido *Unmodified Knesser-ney* (*-ukndiscount*), *Witten-Bell* (*-wbdiscount*) y el descuento aplicado por defecto, *Knesser-Ney*

Como ya es costumbre en este punto, en el anexo se muestra el script desarrollado para automatizar el lanzamiento de los experimentos y en la siguiente tabla se pueden ver los *BLEU* resultantes.

En otra ocasión vemos como ninguno de los experimentos alternativos mejora al modelo original. Se han utilizado todos los modelos con interpolación, puesto que se conoce que funciona mejor que *back-off* para la generación de modelos de n-gramas.

2.6 Ejercicio 6 (opcional)

Para finalizar con la práctica, se proponía experimentar con una variante de *Moses* monótona. Esta modificación se aplica al entrenamiento del modelo de traducción, y se puede realizar fácilmente sustituyendo el comando “msd-bidirectional-fe” por “monotonicity-bidirectional-fe”.

Con esta variación, el *BLEU* que hemos obtenido ha sido: BLEU = 92.27, 97.1/92.7/90.7/88.8 (BP=1.000, ratio=1.000, hyp_len=11785, ref_len=11786). Cómo vemos, este valor supera el obtenido por el modelo base obtenido siguiendo la guía, siendo 0,15 puntos mejor.

3 Conclusiones

La traducción estadística ha sido durante mucho tiempo el estado del arte de la traducción automática. Aunque actualmente las redes neuronales han conseguido mejorar los resultados de la traducción estadística, para ciertos casos en los que no se dispone de grandes cantidades de datos la predicción estadística sigue teniendo un papel reseñable.

En esta práctica más allá de reforzar los aspectos teóricos de la traducción automática basada en modelos estadísticos, hemos revisado y experimentado con una herramienta punta en dicho modo de traducción, así como visto su funcionamiento y posibles modificaciones o experimentaciones que podemos hacer sobre la metodología para ajustarnos mejor a nuestro problema.

Appendices

En el siguiente fragmento de código se muestra el script desarrollado para poder realizar los experimentos de una manera más automática. En concreto la versión que se muestra es la realizada para el ejercicio 5, aunque también podemos ver las modificaciones que hemos realizado en ejercicios anteriores, como la inclusión de la variable de Ngramas.

Solo se va a incluir esta versión para no sobrecargar la memoria con código muy similar en el que solo cambia un parámetro de todo el script, por lo que en cada una de las secciones se ha comentado qué parte es la que se debería cambiar u omitir para obtener el resultado y poder replicar los experimentos.

```
1 export GIZA=/home/miguel/Documentos/TA/mgiza/mgizapp/bin
2 export MOSES=/home/miguel/Documentos/TA/mosesdecoder
3 export SCRIPTS_ROOTDIR=/home/miguel/Documentos/TA/mosesdecoder/
  scripts
4 export PATH=$PATH:/home/miguel/Documentos/TA/mosesdecoder/scripts/
  training/
5 export PATH=$PATH:/home/miguel/Documentos/TA/srilm/bin/i686-m64
6 export PATH=$PATH:/home/miguel/Documentos/TA/mosesdecoder/bin/
7
8 echo $PATH
9
10 echo "global variables exported"
11
12 export MAX_ITER=5
13 export NGRAM=3
14
15 #download train data
16 wget --no-check-certificate \
17 http://www.prhlt.upv.es/~fcu/Students/ta/train.tgz
18 tar xzvf train.tgz
19 rm train.tgz
20
21 echo "Training data downloaded"
22
23 #clean corpus
24 clean-corpus-n.perl train/training es en train/training.clean 1 60
25
26 echo "Training corpus cleaned"
27
28 #train output language model
29 cd train
30 ngram-count -order $NGRAM -unk -interpolate -ukndiscount $NGRAM -
  text training.clean.en -lm model.lm
31 export LangModel=$PWD/model.lm
32
33 echo $LangModel
34 echo "output language model trained!"
35
36 #train translation model
37 $SCRIPTS_ROOTDIR/training/train-model.perl -root-dir work -mgiza -
  mgiza-cpus 1 -corpus training.clean -f es -e en -alignment grow
  -diag-final-and -reordering msd-bidirectional-fe -lm 0:3:
  $LangModel -external-bin-dir $GIZA> training.out
```

```

38 echo "translation model trained!"
39
40 # train log-linear model weights
41 wget --no-check-certificate \
42 http://www.prhlt.upv.es/~fcu/Students/ta/dev.tgz
43 tar xzvf dev.tgz
44 rm dev.tgz
45 mv dev/development.e* .
46 echo "development corpus downloaded"
47
48 clean-corpus-n.perl development es en development.clean 1 60
49 echo "development corpus cleaned"
50
51 cd ..
52 $MOSES/scripts/training/mert-moses.pl \
53 train/development.clean.es train/development.clean.en \
54 $MOSES/bin/moses train/work/model/moses.ini \
55 --maximum-iterations=$MAX_ITER \
56 --mertdir $MOSES/bin/
57
58 echo "log-linear model weights adjusted"
59
60 #translation process
61 wget --no-check-certificate \
62 http://www.prhlt.upv.es/~fcu/Students/ta/test.tgz
63 tar xzvf test.tgz
64 rm test.tgz
65 echo "test data downloaded"
66
67 cd test
68 $MOSES/bin/moses \
69 -f ../mert-work/moses.ini < test.es > test.hyp
70
71 echo "translation performed"
72
73 #evaluation
74 $MOSES/scripts/generic/multi-bleu.perl -lc test.en < test.hyp

```